

PROGRAMACIÓN ORIENTADA A OBJETOS

Unidad: Introducción a la Programación Orientada a Objetos en Python

Trabajo Práctico Integrador — Reserva Natural “El Yaguareté”

Modelar y programar en Python, con clases puras y sin frameworks, el sistema que administra el territorio, la fauna, los circuitos turísticos y el personal de una reserva natural, aplicando de forma explícita asociación con rol, atributos derivados, composición y agregación, herencia (simple, en cadena, overlapping y ortogonal), clases asociación (binaria y ternaria), restricciones {XOR} con superclase deducida, y polimorfismo.

MARCO TEÓRICO

Concepto	Aplicación en el proyecto
Asociación y rol de asociación	La sede administrativa de la Reserva (un Sector que ya existe en la composición) y la relación reflexiva linda_con entre Sectores.
Atributos derivados (@property)	sobrepoblado y en_deficit en Habitat, en_riesgo en Sector, y estado_conservacion en Especie: ninguno se guarda, todos se calculan.
Composición vs. Agregación	Reserva–Sector–Habitat y Circuito–Parada son composición; Circuito–Atraccion es agregación (la atracción preexiste y se comparte).
Herencia simple y en cadena — real vs. clasificación derivada	Especie → Vertebrado → Mamífero/Ave/Reptil es herencia real y fija; estado_conservacion de Especie es clasificación derivada, no una subclase.
Herencia especial: overlapping	Voluntario combina libremente los roles RolCuidado, RolMantenimiento y RolEducacion mediante composición, no herencia múltiple.
Herencia especial: generalización ortogonal	Vehiculo se clasifica por dos ejes independientes: tipo de desplazamiento (Terrestre/Acuático/Aéreo) y forma de propiedad (Propio/Alquilado).
Clase asociación (binaria y ternaria)	CircuitoGuiado reifica la relación Guía–Circuito con su valoración; Avistaje reifica Guía–Especie–Circuito con Turista opcional y diferido.
Restricciones {XOR} y superclase deducida	Solicitante {disjoint, complete} agrupa a Turista y GrupoTuristico; una Expedicion se reserva en modo individual o grupal, nunca ambos.
Polimorfismo	Guia es abstracta con calcular_tarifa_dia(); GuiaLocal y GuiaCertificado {disjoint, complete} lo implementan cada una a su manera, sin condicionales de tipo.

CASO PRÁCTICO

Enunciado general

La Reserva Natural “El Yaguararé” pidió modelar y programar el sistema que administra su territorio, su fauna, sus circuitos turísticos y su personal. A continuación se transcribe el relevamiento realizado con la dirección de la reserva:

“La reserva está organizada en sectores, y cada sector agrupa varios hábitats. Un sector no existe si no pertenece a una reserva, y un hábitat no existe si no pertenece a un sector. La reserva tiene, además, un sector que funciona como sede administrativa — uno de los sectores que ya forman parte de la reserva. Los sectores, a su vez, lindan entre sí: si el sector Norte linda con el sector Centro, el sector Centro linda con el sector Norte.

Cada hábitat tiene una población animal actual y una capacidad máxima; se considera sobrepoblado cuando la población supera la capacidad. También tiene un presupuesto de alimentación y un costo de mantenimiento; está en déficit cuando el costo supera el presupuesto. Un sector está en riesgo cuando, de sus hábitats sobrepoblados, la mitad o más está además en déficit.

La reserva ofrece circuitos turísticos. Un circuito se arma con paradas — cada parada es un tramo entre dos puntos del circuito y no tiene sentido fuera de él — y puede incluir atracciones ya existentes en la reserva, como miradores o aguadas, que siguen existiendo aunque el circuito se elimine y pueden formar parte de más de un circuito a la vez.

Las especies que habitan la reserva se clasifican taxonómicamente: toda especie es un vertebrado, y todo vertebrado es un mamífero, un ave o un reptil — una clasificación que nunca cambia para una especie dada. Además, cada especie tiene un estado de conservación (en peligro, vulnerable o preocupación menor) que depende de su población actual frente a un umbral crítico definido para la especie, y que puede cambiar con el tiempo a medida que la población varía.

La reserva cuenta con voluntarios que pueden combinar libremente distintos roles: cuidado de fauna, mantenimiento de senderos y educación ambiental. Un mismo voluntario puede desempeñar los tres roles a la vez, dos de ellos, uno solo, o sumar y dejar roles con el tiempo.

Para patrullar el territorio, la reserva dispone de vehículos que se clasifican por dos criterios independientes entre sí: por su tipo de desplazamiento (terrestre, acuático o aéreo) y, por separado, por su forma de propiedad (propio o alquilado). Un vehículo terrestre puede ser propio o alquilado exactamente igual que uno aéreo; cambiar un criterio no afecta al otro.

Cada guía ofrece uno o más circuitos, y para cada circuito que ofrece registra una valoración de 1 a 10 que indica qué tan bien lo conoce. La misma persona guía puede tener valoraciones distintas para circuitos distintos, y el mismo circuito puede tener valoraciones distintas según qué guía lo ofrezca.

Durante los circuitos guiados se registran avistajes de fauna: cada avistaje queda ligado al guía que lo condujo, a la especie avistada, al circuito por el que se realizó, e incluye la fecha y la cantidad de ejemplares avistados. Un avistaje puede completarse después con el turista que lo reportó personalmente — pero solo si ese turista integra la expedición que reservó ese circuito en esa fecha.

Los guías pueden ser guías locales o guías certificados — nunca ambas cosas — y ambos deben poder calcular su tarifa por día de trabajo. Un guía local cobra su tarifa base por los días trabajados. Un guía certificado cobra la misma tarifa base... y además un plus por cada idioma adicional que domina.

Por último, la reserva gestiona expediciones: cada expedición reserva un circuito para una fecha determinada, y puede reservarla un turista individual o un grupo turístico — pero nunca de las dos formas a la vez para la misma expedición. Tanto el turista individual como el grupo turístico participan de la reserva del mismo modo frente a la expedición: ambos tienen una cantidad de personas y un contacto responsable.”

Desarrollo del proyecto

Siguiendo los pasos indicados:

- a) Analizar el enunciado e identificar, para cada clase y cada relación, cuál de los 8 conceptos del marco teórico aplica. Documentar esas decisiones en el README (no hace falta UML formal; alcanza con una lista o un diagrama simple).
- b) Crear el proyecto en Python puro, sin frameworks, organizado en módulos o paquetes (por ejemplo: **reserva/**, **fauna/**, **personal/**, **turismo/**, **vehiculos/**).
- c) Implementar la composición **Reserva** → **Sector** → **Habitat**, la asociación reflexiva **linda_con** entre sectores y el rol **sede** de la Reserva hacia uno de sus propios sectores.
- d) Implementar los atributos derivados de **Habitat** (sobrepoblado, en_deficit) y de **Sector** (en_riesgo) con **@property**, sin guardarlos como atributos comunes.
- e) Implementar **Circuito** con composición hacia **Parada** y agregación hacia **Atraccion**, demostrando que al eliminar un circuito sus paradas dejan de existir pero sus atracciones sobreviven.
- f) Implementar la jerarquía taxonómica de **Especie** (Vertebrado → Mamifero / Ave / Reptil) como herencia real, y **estado_conservacion** como atributo derivado, nunca como subclase.
- g) Implementar **Voluntario** con roles combinables (**RolCuidado**, **RolMantenimiento**, **RolEducacion**) mediante composición, no herencia múltiple.
- h) Implementar **Vehiculo** con generalización ortogonal: eje tipo (Terrestre/Acuático/Aéreo) y eje propiedad (Propio/Alquilado), modelados por composición.
- i) Implementar **CircuitoGuiado** como clase asociación binaria entre **Guia** y **Circuito**, con el atributo **valoracion**.
- j) Implementar **Avistaje** como clase asociación ternaria entre **Guia**, **Especie** y **Circuito**, con **Turista** opcional y diferido, validando que pertenezca a la expedición correspondiente.
- k) Implementar **Solicitante** como superclase deducida {disjoint, complete} de **Turista** y **GrupoTuristico**, y la restricción {XOR} entre la reserva individual y grupal de una **Expedicion**.
- l) Implementar **Guia** como clase abstracta con el método abstracto **calcular_tarifa_dia()**, y **GuiaLocal** / **GuiaCertificado** como subclases {disjoint, complete} que lo implementan sin condicionales de tipo en el código que las usa.
- m) Escribir un script **demo.py** que instancie el modelo completo y muestre por consola el resultado de cada concepto aplicado (ver HU-11).
- n) Documentar en un **README.md** la estructura del proyecto y las decisiones de diseño de cada uno de los 8 conceptos.

HISTORIAS DE USUARIO

Las siguientes historias de usuario describen, desde la perspectiva de quien desarrolla el sistema, el comportamiento esperado del modelo. Cada historia sigue el formato “Como... quiero... para...” y se acompaña de sus criterios de aceptación.

HU-01 — Modelar el territorio de la reserva con composición y rol de asociación

Como desarrollador, quiero modelar la Reserva, sus Sectores y sus Hábitats con composición en cadena, además de la sede administrativa y la relación reflexiva entre sectores, para representar fielmente la estructura territorial.

Criterios de aceptación:

- Reserva, Sector y Habitat forman una composición en cadena: eliminar una Reserva elimina sus Sectores, y eliminar un Sector elimina sus Hábitats.
- Reserva.sede referencia a un Sector que ya pertenece a esa misma Reserva; no crea un sector nuevo.
- La relación `linda_con` entre Sectores es reflexiva y bidireccional.

HU-02 — Calcular indicadores de población y presupuesto con atributos derivados

Como desarrollador, quiero que Habitat y Sector calculen sus indicadores con `@property` en lugar de guardarlos, para que nunca queden desincronizados de los datos reales.

Criterios de aceptación:

- `Habitat.sobrepoblado` y `Habitat.en_deficit` están implementados con `@property`, sin atributo guardado equivalente.
- `Sector.en_riesgo` filtra los hábitats sobrepoblados y evalúa cuántos de ellos están además en déficit.
- Ningún atributo derivado se asigna manualmente en `__init__`.

HU-03 — Distinguir composición y agregación en los circuitos turísticos

Como desarrollador, quiero que Circuito trate a sus Paradas como composición y a sus Atracciones como agregación, para reflejar que unas dependen del circuito y otras no.

Criterios de aceptación:

- Un Circuito tiene 2 o más Paradas que se destruyen junto con el circuito.
- Un Circuito puede incluir 0 o más Atracciones preexistentes que sobreviven si el circuito se elimina.
- Una misma Atracción puede pertenecer a más de un Circuito a la vez.

HU-04 — Modelar la taxonomía de especies y su estado de conservación

Como desarrollador, quiero modelar Especie con una jerarquía taxonómica fija y calcular su estado de conservación de forma dinámica, para no confundir un tipo que nunca cambia con una categoría que sí puede cambiar.

Criterios de aceptación:

- Vertebrado hereda de Especie; Mamífero, Ave y Reptil heredan de Vertebrado.
- Ninguna instancia cambia de subclase en tiempo de ejecución.
- `estado_conservacion` es una `@property` que depende de `poblacion_actual` y `umbral_critico`, nunca una subclase.

HU-05 — Modelar voluntarios con roles combinables (herencia overlapping)

Como desarrollador, quiero que un Voluntario pueda combinar libremente roles de cuidado, mantenimiento y educación ambiental, para reflejar que una misma persona puede sumar o dejar roles con el tiempo.

Criterios de aceptación:

- Voluntario no hereda de `RolCuidado`, `RolMantenimiento` ni `RolEducacion`.
- Voluntario compone una colección de 0 a 3 roles, agregables y quitables en cualquier momento.
- Es posible que un mismo Voluntario tenga los tres roles simultáneamente.

HU-06 — Modelar vehículos de patrullaje con generalización ortogonal

Como desarrollador, quiero clasificar los vehículos por tipo de desplazamiento y por forma de propiedad de manera independiente, para que un cambio en un eje no afecte al otro.

Criterios de aceptación:

- Vehículo se modela por composición, con un detalle de tipo y un detalle de propiedad.
- Cada eje es {disjoint, complete}: todo vehículo tiene exactamente un valor por eje.
- Cambiar el valor de un eje no modifica el valor del otro eje del mismo vehículo.

HU-07 — Reificar la relación entre guías y circuitos

Como desarrollador, quiero que la valoración de un guía sobre un circuito se registre como una clase asociación, para que cada combinación guía–circuito tenga su propio dato sin duplicar información.

Criterios de aceptación:

- CircuitoGuiado conecta a Guía y Circuito, y tiene su propio atributo valoracion (1 a 10).
- Un mismo Guía puede tener distinta valoración para distintos Circuitos.
- Un mismo Circuito puede tener distinta valoración según qué Guía lo ofrezca.

HU-08 — Registrar avistajes como clase asociación ternaria

Como desarrollador, quiero registrar cada avistaje ligando guía, especie y circuito, con la posibilidad de sumar después al turista que lo reportó, para llevar un historial preciso de fauna observada.

Criterios de aceptación:

- Avistaje liga obligatoriamente a un Guía, una Especie y un Circuito, más fecha y cantidad avistada.
- El Turista de un Avistaje es opcional (0..1) y se puede agregar después de creado el registro.
- No se puede asignar a un Avistaje un Turista que no pertenezca a la Expedicion de ese circuito en esa fecha.

HU-09 — Modelar expediciones con superclase deducida y restricción {XOR}

Como desarrollador, quiero que una Expedicion pueda reservarse por un turista individual o por un grupo turístico, nunca ambos a la vez, para respetar cómo la reserva realmente vende sus circuitos.

Criterios de aceptación:

- Solicitante es una superclase deducida {disjoint, complete} de Turista y GrupoTuristico.
- Toda Expedicion tiene exactamente un Solicitante.
- El código impide crear una Expedicion con Turista y GrupoTuristico asignados a la vez.

HU-10 — Calcular tarifas de guías con polimorfismo real

Como desarrollador, quiero que cada guía calcule su propia tarifa por día sin que el código que las usa pregunte de qué subclase se trata, para que el sistema crezca sin modificar código existente.

Criterios de aceptación:

- Guía es una clase abstracta con el método abstracto calcular_tarifa_dia().
- GuíaLocal y GuíaCertificado son {disjoint, complete} e implementan calcular_tarifa_dia() cada una a su manera.
- El código que suma tarifas de una lista mixta de guías no usa isinstance ni condicionales por tipo.

HU-11 — Demostración integradora del modelo completo

Como desarrollador, quiero un script único que instancie todo el modelo y muestre por consola el resultado de cada concepto aplicado, para verificar que el sistema funciona de punta a punta.

Criterios de aceptación:

- demo.py instancia al menos: 1 Reserva con 2 Sectores y 2 Hábitats cada uno, 3 Especies de clases taxonómicas distintas, 2 Voluntarios con roles distintos, 2 Vehículos en ejes distintos, 1 GuíaLocal y 1 GuíaCertificado, 1 CircuitoGuiado, 2 Avistajes (uno con Turista y otro sin) y 1 Expedicion individual y 1 grupal.
- La consola muestra el resultado de cada atributo derivado y de cada llamado polimórfico.
- El script corre sin errores con python demo.py.

REGLAS DE NEGOCIO

Las reglas de negocio definen las restricciones que el modelo debe hacer cumplir para mantener la integridad y la coherencia del dominio. Complementan a las historias de usuario: una historia describe qué se quiere lograr; una regla de negocio establece qué está permitido y qué debe rechazarse.

ID	Regla de negocio	Entidad / Verificación
RN-01	Reserva → Sector → Habitat es composición en cadena: eliminar el todo elimina las partes en cada nivel.	Reserva / Sector / Habitat
RN-02	Reserva.sede debe apuntar a un Sector que ya pertenezca a esa Reserva; no se admite un Sector externo.	Reserva / sede
RN-03	La relación linda_con entre Sectores es reflexiva y bidireccional: agregar A→B implica agregar B→A.	Sector / linda_con
RN-04	Habitat.sobrepoblado y Habitat.en_deficit se implementan exclusivamente como @property.	Habitat / atributos derivados
RN-05	Sector.en_riesgo es verdadero cuando la mitad o más de sus hábitats sobrepoblados está también en déficit.	Sector / en_riesgo
RN-06	Circuito–Parada es composición (mínimo 2 paradas, no sobreviven fuera del circuito); Circuito–Atraccion es agregación (la atracción preexiste y puede compartirse entre circuitos).	Circuito / Parada / Atraccion
RN-07	La jerarquía Especie → Vertebrado → {Mamifero, Ave, Reptil} es fija: ninguna instancia cambia de subclase.	Especie / herencia
RN-08	Especie.estado_conservacion se calcula comparando poblacion_actual con umbral_critico; no se modela como subclase.	Especie / estado_conservacion
RN-09	Voluntario admite entre 0 y 3 roles simultáneos (RolCuidado, RolMantenimiento, RolEducacion) modelados por composición.	Voluntario / roles
RN-10	Vehiculo se clasifica por dos ejes independientes — tipo (Terrestre/Acuático/Aéreo) y propiedad (Propio/Alquilado) — cada uno {disjoint, complete}.	Vehiculo / generalización ortogonal
RN-11	CircuitoGuiado registra una valoración de 1 a 10 por cada combinación Guia–Circuito; cada combinación genera su propia instancia.	CircuitoGuiado / valoracion
RN-12	Avistaje exige Guia (1), Especie (1) y Circuito (1); Turista es 0..1 y se agrega después de creado el avistaje.	Avistaje / multiplicidad
RN-13	El Turista agregado a un Avistaje debe pertenecer a la Expedicion que reservó ese Circuito en esa fecha.	Avistaje / integridad referencial
RN-14	Solicitante {disjoint, complete} agrupa a Turista y GrupoTuristico; toda Expedicion tiene exactamente un Solicitante.	Expedicion / Solicitante
RN-15	Restricción {XOR}: una Expedicion no puede tener asignados Turista y GrupoTuristico al mismo tiempo.	Expedicion / XOR
RN-16	Guia es abstracta y no instanciable; GuiaLocal y GuiaCertificado son {disjoint, complete} e implementan calcular_tarifa_dia() sin condicionales de tipo en el código cliente.	Guia / polimorfismo

CONCLUSIONES ESPERADAS

Al finalizar el trabajo práctico, el estudiante debe demostrar:

- Relaciones entre clases correctamente diferenciadas: asociación con rol, relaciones reflexivas, composición y agregación.
- Uso consistente de atributos derivados (@property) para todo dato que se calcula a partir de otros, nunca guardado como atributo simple.
- Herencia bien aplicada: jerarquías reales (fijas) separadas de clasificaciones derivadas (calculadas), y herencia especial (overlapping y ortogonal) resuelta por composición, no por herencia múltiple.
- Clases asociación binaria y ternaria que reifican relaciones con datos propios.
- Restricciones de dominio correctamente implementadas: superclases deducidas y restricciones {XOR}.
- Polimorfismo real: método abstracto con override, sin condicionales de tipo (isinstance) en el código cliente.
- Buenas prácticas de Python: proyecto modular, docstrings, nombres consistentes con el dominio, sin frameworks externos.

MODALIDAD DE ENTREGA

- Proyecto en Python puro (sin frameworks ni librerías externas más allá de la biblioteca estándar).
- Organización en módulos o paquetes según el dominio (por ejemplo reserva/, fauna/, personal/, turismo/, vehiculos/).
- Incluir demo.py que instancie el modelo completo y muestre resultados por consola.
- Incluir README.md con la estructura del proyecto y una breve justificación de cada uno de los 8 conceptos aplicados.
- Eliminar entornos virtuales (venv/) y caché (__pycache__/) antes de comprimir.
- Comprimir el proyecto en un único archivo .zip.
- Al descomprimir, python demo.py debe correr sin errores.